```
  private static void initializeSieve(int maxValue)
  {
    // declarations
    s = maxValue + 1; // size of array
    f = new boolean[s];
    int i;

    // initialize array to true.
    for (i = 0; i < s; i++)
      f[i] = true;

    // get rid of known non-primes
    f[0] = f[1] = false;
  }
}
```

The `initializeSieve` function is a little messy, so in Listing 5–4, I cleaned it up considerably. First, I replaced all usages of the `s` variable with `f.length`. Then, I changed the names of the three functions to something a bit more expressive. Finally, I rearranged the innards of `initializeArrayOfIntegers` (née `initializeSieve`) to be a little nicer to read. The tests all still ran.

Listing 5-4

PrimeGenerator.java, version 3 (partial)

```
public class PrimeGenerator
{
  private static boolean[] f;
  private static int[] result;

  public static int[] generatePrimes(int maxValue)
  {
    if (maxValue < 2)
      return new int[0];
    else
    {
      initializeArrayOfIntegers(maxValue);
      crossOutMultiples();
      putUncrossedIntegersIntoResult();
      return result;
    }
  }

  private static void initializeArrayOfIntegers(int maxValue)
  {
    f = new boolean[maxValue + 1];
    f[0] = f[1] = false;  //neither primes nor multiples.
    for (int i = 2; i < f.length; i++)
      f[i] = true;
  }
```

Next, I looked at `crossOutMultiples`. There were a number of statements in this function, and in others, of the form `if(f[i] == true)`. The intent was to check to see if `i` was uncrossed, so I changed the name of `f` to `unCrossed`. But this lead to ugly statements like `unCrossed[i] = false`. I found the double negative confusing. So I changed the name of the array to `isCrossed` and changed the sense of all the booleans. The tests all still ran.

I got rid of the initialization that set `isCrossed[0]` and `isCrossed[1]` to `true` and just made sure that no part of the function used the `isCrossed` array for indexes less than 2. I extracted the inner loop of the `crossOutMultiples` function and called it `crossOutMultiplesOf`. I also thought that `if(isCrossed[i] == false)` was confusing, so I created a function called `notCrossed` and changed the `if` statement to `if (notCrossed(i))`. The tests all still ran.

I spent a bit of time writing a comment that tried to explain why you only have to iterate up to the square root of the array size. This led me to extract the calculation into a function, where I could put the explanatory comment. In writing the comment, I realized that the square root is the maximum prime factor of any integer in the array. So I chose that name for the variables and functions that dealt with it. The result of all these refactorings are in Listing 5–5. The tests all still ran.

Listing 5-5

PrimeGenerator.java version 4 (partial)

```
public class PrimeGenerator
{
  private static boolean[] isCrossed;
  private static int[] result;

  public static int[] generatePrimes(int maxValue)
  {
    if (maxValue < 2)
      return new int[0];
    else
    {
      initializeArrayOfIntegers(maxValue);
      crossOutMultiples();
      putUncrossedIntegersIntoResult();
      return result;
    }
  }

  private static void initializeArrayOfIntegers(int maxValue)
  {
    isCrossed = new boolean[maxValue + 1];
    for (int i = 2; i < isCrossed.length; i++)
      isCrossed[i] = false;
  }

  private static void crossOutMultiples()
  {
    int maxPrimeFactor = calcMaxPrimeFactor();
    for (int i = 2; i <= maxPrimeFactor; i++)
      if (notCrossed(i))
        crossOutMultiplesOf(i);
  }

  private static int calcMaxPrimeFactor()
  {
    // We cross out all multiples of p, where p is prime.
    // Thus, all crossed out multiples have p and q for
    // factors.  If p > sqrt of the size of the array, then
    // q will never be greater than 1.  Thus p is the
    // largest prime factor in the array, and is also
    // the iteration limit.
```

```
    double maxPrimeFactor = Math.sqrt(isCrossed.length) + 1;
    return (int) maxPrimeFactor;
  }

  private static void crossOutMultiplesOf(int i)
  {
    for (int multiple = 2*i;
         multiple < isCrossed.length;
         multiple += i)
      isCrossed[multiple] = true;
  }

  private static boolean notCrossed(int i)
  {
    return isCrossed[i] == false;
  }
```

The last function to refactor is `putUncrossedIntegersIntoResult`. This method has two parts. The first counts the number of uncrossed integers in the array and creates the result array of that size. The second moves the uncrossed integers into the result array. I extracted the first part into its own function and did some miscellaneous cleanup. The tests all still ran.

Listing 5-6

PrimeGenerator.java, version 5 (partial)

```
private static void putUncrossedIntegersIntoResult()
  {
    result = new int[numberOfUncrossedIntegers()];
    for (int j = 0, i = 2; i < isCrossed.length; i++)
      if (notCrossed(i))
        result[j++] = i;
  }

  private static int numberOfUncrossedIntegers()
  {
    int count = 0;
    for (int i = 2; i < isCrossed.length; i++)
      if (notCrossed(i))
        count++;

    return count;
  }
```

The Final Reread



Next, I made one final pass over the whole program, reading it from beginning to end, rather like one would read a geometric proof. This is an important step. So far, I've been refactoring fragments. Now, I want to see if the whole program hangs together as a *readable* whole.

First, I realize that I don't like the name `initializeArrayOfIntegers`. What's being initialized is not, in fact, an array of integers, it's an array of booleans. However, `initializeArrayOfBooleans` is not an improvement. What we are really doing in this method is uncrossing all the relevant integers so that we can then cross out the multiples. So I change the name to `uncrossIntegersUpTo`. I also realize that I don't like the name `isCrossed` for the array of booleans. So I change it to `crossedOut`. The tests all still run.

One might think I'm being frivolous with these name changes, but with a refactoring browser you can afford to do these kinds of tweaks—they cost virtually nothing. Even without a refactoring browser, a simple search and replace is pretty cheap. And the tests strongly mitigate any chance that we might unknowingly break something.

I don't know what I was smoking when I wrote all that `maxPrimeFactor` stuff. Yikes! The square root of the size of the array is not necessarily prime. That method did *not* calculate the maximum prime factor. The explanatory comment was just *wrong*. So I rewrote the comment to better explain the rationale behind the square root and renamed all the variables appropriately.[3] The tests all still run.

What the devil is that +1 doing in there? I think it must have been paranoia. I was afraid that a fractional square root would convert to an integer that was too small to serve as the iteration limit. But that's silly. The true iteration limit is the largest prime less than or equal to the square root of the size of the array. I'll get rid of the +1.

The tests all run, but that last change makes me pretty nervous. I understand the rationale behind the square root, but I've got a nagging feeling that there may be some corner cases that aren't being covered. So I'll write another test to check that there are no multiples in any of the prime lists between 2 and 500. (See the `testExhaustive` function in Listing 5–8.) The new test passes, and my fears are allayed.

The rest of the code reads pretty nicely. So I think we're done. The final version is shown in Listings 5–7 and 5–8.

Listing 5-7

PrimeGenerator.java (final)

```
/**
 * This class generates prime numbers up to a user specified
 * maximum.  The algorithm used is the Sieve of Eratosthenes.
 * Given an array of integers starting at 2:
 * Find the first uncrossed integer, and cross out all its
 * multiples.  Repeat until there are no more multiples
 * in the array.
 */

public class PrimeGenerator
{
  private static boolean[] crossedOut;
  private static int[] result;

  public static int[] generatePrimes(int maxValue)
  {
    if (maxValue < 2)
      return new int[0];
    else
    {
      uncrossIntegersUpTo(maxValue);
      crossOutMultiples();
      putUncrossedIntegersIntoResult();
      return result;
    }
  }

  private static void uncrossIntegersUpTo(int maxValue)
  {
```

3. When Kent Beck and Jim Newkirk refactored this program, they did away with the square root altogether. Kent's rationale was that the square root was hard to understand, and there was no test that failed if you iterated right up to the size of the array. I can't bring myself to give up the efficiency. I guess that shows my assembly-language roots.

```
    crossedOut = new boolean[maxValue + 1];
    for (int i = 2; i < crossedOut.length; i++)
      crossedOut[i] = false;
  }

  private static void crossOutMultiples()
  {
    int limit = determineIterationLimit();
    for (int i = 2; i <= limit; i++)
      if (notCrossed(i))
        crossOutMultiplesOf(i);
  }

  private static int determineIterationLimit()
  {
    // Every multiple in the array has a prime factor that
    // is less than or equal to the sqrt of the array size,
    // so we don't have to cross out multiples of numbers
    // larger than that root.
    double iterationLimit = Math.sqrt(crossedOut.length);
    return (int) iterationLimit;
  }

  private static void crossOutMultiplesOf(int i)
  {
    for (int multiple = 2*i;
         multiple < crossedOut.length;
         multiple += i)
      crossedOut[multiple] = true;
  }

  private static boolean notCrossed(int i)
  {
    return crossedOut[i] == false;
  }

  private static void putUncrossedIntegersIntoResult()
  {
    result = new int[numberOfUncrossedIntegers()];
    for (int j = 0, i = 2; i < crossedOut.length; i++)
      if (notCrossed(i))
        result[j++] = i;
  }

  private static int numberOfUncrossedIntegers()
  {
    int count = 0;
    for (int i = 2; i < crossedOut.length; i++)
      if (notCrossed(i))
        count++;

    return count;
  }
}
```

Listing 5-8

TestGeneratePrimes.java (final)

```
import junit.framework.*;

public class TestGeneratePrimes extends TestCase
{
  public static void main(String args[])
  {
      junit.swingui.TestRunner.main(
        new String[] {"TestGeneratePrimes"});
  }
  public TestGeneratePrimes(String name)
  {
    super(name);
  }

  public void testPrimes()
  {
    int[] nullArray = PrimeGenerator.generatePrimes(0);
    assertEquals(nullArray.length, 0);

    int[] minArray = PrimeGenerator.generatePrimes(2);
    assertEquals(minArray.length, 1);
    assertEquals(minArray[0], 2);

    int[] threeArray = PrimeGenerator.generatePrimes(3);
    assertEquals(threeArray.length, 2);
    assertEquals(threeArray[0], 2);
    assertEquals(threeArray[1], 3);

    int[] centArray = PrimeGenerator.generatePrimes(100);
    assertEquals(centArray.length, 25);
    assertEquals(centArray[24], 97);
  }

  public void testExhaustive()
  {
    for (int i = 2; i<500; i++)
        verifyPrimeList(PrimeGenerator.generatePrimes(i));
  }

  private void verifyPrimeList(int[] list)
  {
    for (int i=0; i<list.length; i++)
        verifyPrime(list[i]);
  }

  private void verifyPrime(int n)
  {
    for (int factor=2; factor<n; factor++)
      assert(n%factor != 0);
  }
}
```

Conclusion

The end result of this program reads much better than it did at the start. The program also works a bit better. I'm pretty pleased with the outcome. The program is much easier to understand and is therefore much easier to change. Also, the structure of the program has isolated its parts from one another. This also makes the program much easier to change.

You might be worried that extracting functions that are only called once might adversely affect performance. I think the increased readability is worth a few extra nanoseconds in most cases. However, there may be deep inner loops where those few nanoseconds will be costly. My advice is to assume that the cost will be negligible and wait to be proven wrong.

Was this worth the time we invested in it? After all, the function worked when we started. I strongly recommend that you *always* practice such refactoring for *every* module you write and for *every* module you maintain. The time investment is very small compared to the effort you'll be saving yourself and others in the near future.

Refactoring is like cleaning up the kitchen after dinner. The first time you skip it, you are done with dinner more quickly. But that lack of clean dishes and clear working space makes dinner take longer to prepare the next day. This makes you want to skip cleaning again. Indeed, you can always finish dinner faster *today* if you skip cleaning, but the mess builds and builds. Eventually you are spending an inordinate amount of time hunting for the right cooking utensils, chiseling the encrusted dried food off the dishes, and scrubbing them down so that they are suitable to cook with. Dinner takes forever. Skipping the cleanup does not really make dinner go faster.

The goal of refactoring, as depicted in this chapter, is to clean your code every day. We don't want the mess to build. We don't want to have to chisel and scrub the encrusted bits that accumulate over time. We want to be able to extend and modify our system with a minimum of effort. The most important enabler of that ability is the cleanliness of the code.

I can't stress this enough. All the principles and patterns in this book come to naught if the code they are employed within is a mess. Before investing in principles and patterns, invest in clean code.

Bibliography

1. Fowler, Martin. *Refactoring: Improving the Design of Existing Code*. Reading, MA: Addison–Wesley, 1999.

6

A Programming Episode



Design and programming are human activities; forget that and all is lost.

—Bjarne Stroustrup, 1991

In order to demonstrate the XP programming practices, Bob Koss (RSK) and Bob Martin (RCM) will pair program a simple application while you watch like a fly on the wall. We will use test-driven development and a lot of refactoring to create our application. What follows is a pretty faithful reenactment of a programming episode that the two Bobs actually did in a hotel room in late 2000.

We made lots of mistakes while doing this. Some of the mistakes are in code, some are in logic, some are in design, and some are in requirements. As you read, you will see us flail around in all these areas, identifying and then dealing with our errors and misconceptions. The process is messy—as are all human processes. The result ... well, it's amazing the order that can arise out of such a messy process.

The program calculates the score of a game of bowling, so it helps if you know the rules. If you don't know the rules of bowling, then check out the accompanying sidebar.

The Bowling Game

RCM: Will you help me write a little application that calculates bowling scores?

RSK: (Reflects to himself, "The XP practice of pair programming says that I can't say, "no," when asked to help. I suppose that's especially true when it is your boss who is asking.") Sure, Bob, I'd be glad to help.

RCM: OK, great! What I'd like to do is write an application that keeps track of a bowling league. It needs to record all the games, determine the ranks of the teams, determine the winners and losers of each weekly match, and accurately score each game.

RSK: Cool. I used to be a pretty good bowler. This will be fun. You rattled off several user stories, which one would you like to start with?

RCM: Let's begin with scoring a single game.

RSK: Okay. What does that mean? What are the inputs and outputs for this story?

RCM: It seems to me that the inputs are simply a sequence of throws. A throw is just an integer that tells how many pins were knocked down by the ball. The output is just the score for each frame.

RSK: I'm assuming you are acting as the customer in this exercise, so what form do you want the inputs and outputs to be in?

RCM: Yes, I'm the customer. We'll need a function to call to add throws and another function that gets the score. Sort of like

```
throwBall(6);
throwBall(3);
assertEquals(9, getScore());
```

RSK: OK, we're going to need some test data. Let me sketch out a little picture of a score card. (See Figure 6-1.)

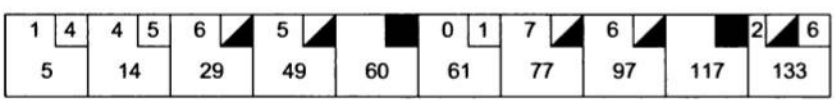
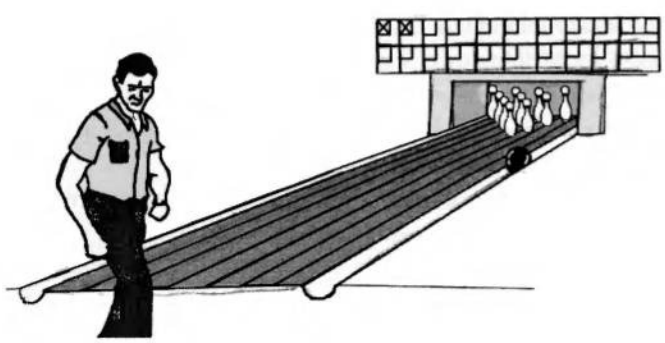


Figure 6-1 Typical Bowling Score Card

RCM: That guy is pretty erratic.

RSK: Or drunk, but it will serve as a decent acceptance test.



RCM: We'll need others, but let's deal with that later. How should we start? Shall we come up with a design for the system?

RSK: I wouldn't mind a UML diagram showing the problem domain concepts that we might see from the score card. That will give us some candidate objects that we can explore further in code.

RCM: (putting on his powerful object designer hat) OK, clearly a `Game` object consists of a sequence of ten frames. Each `Frame` object contains one, two, or three throws.

RSK: Great minds. That was exactly what I was thinking. Let me quickly draw that. (See Figure 6-2.)

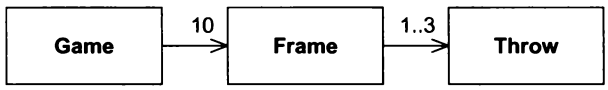


Figure 6-2 UML Diagram of Bowling Score Card

RSK: Well, pick a class ... any class. Shall we start at the end of the dependency chain and work backwards? That will make testing easier.

RCM: Sure, why not. Let's create a test case for the `Throw` class.

RSK: (Starts typing.)

```
//TestThrow.java---------------------------------
import junit.framework.*;

public class TestThrow extends TestCase
{
  public TestThrow(String name)
  {
    super(name);
  }

// public void test????
}
```

RSK: Do you have a clue what the behavior of a `Throw` object should be?

RCM: It holds the number of pins knocked down by the player.

RSK: Okay, you just said, in not so many words, that it doesn't really do anything. Maybe we should come back to it and focus on an object that actually has behavior, instead of one that's just a data store.

RCM: Hmm. You mean the `Throw` class might not really exist?

RSK: Well, if it doesn't have any behavior, how important can it be? I don't know if it exists or not yet. I'd just feel more productive if we were working on an object that had more than setters and getters for methods. But if you want to drive ... (slides the keyboard to RCM).

RCM: Well, let's move up the dependency chain to `Frame` and see if there are any test cases we can write that will force us to finish `Throw`. (Pushes the keyboard back to RSK.)

RSK: (Wondering if RCM is leading me down a blind alley to educate me or if he is really agreeing with me.) Okay, new file, new test case.

```
//TestFrame.java---------------------------------
import junit.framework.*;
```

```
public class TestFrame extends TestCase
{
  public TestFrame( String name )
  {
    super( name );
  }
  //public void test???
}
```

RCM: OK, that's the second time we've typed that. Now, can you think of any interesting test cases for `Frame`?

RSK: A `Frame` might provide its score, the number of pins on each throw, whether there was a strike or a spare . . .

RCM: OK, show me the code.

RSK: (types)

```
//TestFrame.java---------------------------------
import junit.framework.*;

public class TestFrame extends TestCase
{
  public TestFrame( String name )
  {
    super( name );
  }

 public void testScoreNoThrows()
 {
  Frame f = new Frame();
  assertEquals( 0, f.getScore() );
 }
}

//Frame.java-----------------------------------------
public class Frame
{
  public int getScore()
  {
    return 0;
  }
}
```

RCM: OK, the test case passes, but `getScore` is a really stupid function. It will fail if we add a throw to the `Frame`. So let's write the test case that adds some throws and then checks the score.

```
//TestFrame.java---------------------------------

 public void testAddOneThrow()
 {
   Frame f = new Frame();
   f.add(5);
   assertEquals(5, f.getScore());
 }
```

RCM: That doesn't compile. There's no add method in `Frame`.

RSK: I'll bet if you define the method, it will compile ;-)

RCM:

```
//Frame.java-----------------------------------------
public class Frame
{
  public int getScore()
  {
    return 0;
  }

  public void add(Throw t)
  {
  }
}
```

RCM: (thinking out loud) This doesn't compile because we haven't written the `Throw` class.

RSK: Talk to me, Bob. The test is passing an integer, and the method expects a `Throw` object. You can't have it both ways. Before we go down the `Throw` path again, can you describe its behavior?

RCM: Wow! I didn't even notice that I had written `f.add(5)`. I should have written `f.add(new Throw(5))`, but that's ugly as hell. What I *really* want to write is `f.add(5)`.

RSK: Ugly or not, let's leave aesthetics out of it for the time being. Can you describe any behavior of a `Throw` object—binary response, Bob?

RCM: 101101011010100101. I don't know if there is any behavior in `Throw`. I'm beginning to think a `Throw` is just an `int`. However, we don't need to consider that yet since we can write `Frame.add` to take an `int`.

RSK: Then I think we should do that for no other reason than it's simple. When we feel pain, we can do something more sophisticated.

RCM: Agreed.

```
//Frame.java-----------------------------------------
public class Frame
{
  public int getScore()
  {
    return 0;
  }

  public void add(int pins)
  {
  }
}
```

RCM: OK, this compiles and fails the test. Now, let's make the test pass.

```
//Frame.java-----------------------------------------
public class Frame
{
  public int getScore()
```

```
    {
      return itsScore;
    }

      public void add(int pins)
    {
      itsScore += pins;
    }
    private int itsScore = 0;
}
```

RCM: This compiles and passes the tests, but it's clearly simplistic. What's the next test case?

RSK: Can we take a break first?

----------Break----------

RCM: That's better.

`Frame.add` is a fragile function. What if you call it with an 11?

RSK: It can throw an exception if that happens. But, who is calling it? Is this going to be an application framework that thousands of people will use and we have to protect against such things, or is this going to be used by you and only you? If the latter, just don't call it with an 11 (chuckle).

RCM: Good point, the tests in the rest of the system will catch an invalid argument. If we run into trouble, we can put the check in later.

So, the `add` function doesn't currently handle strikes or spares. Let's write a test case that expresses that.

RSK: Hmmmm . . . if we call `add(10)` to represent a strike, what should `getScore()` return? I don't know how to write the assertion, so maybe we're asking the wrong question. Or we're asking the right question to the wrong object.

RCM: When you call `add(10)`, or `add(3)` followed by `add(7)`, then calling `getScore` on the `Frame` is meaningless. The `Frame` would have to look at later `Frame` instances to calculate its score. If those later `Frame` instances don't exist, then it would have to return something ugly, like `-1`. I don't want to return `-1`.

RSK: Yeah, I hate the `-1` idea too. You've introduced the idea of `Frames` knowing about other `Frames`. Who is holding these different `Frame` objects?

RCM: The `Game` object.

RSK: So `Game` depends on `Frame`; and `Frame`, in turn, depends back on `Game`. I hate that.

RCM: `Frames` don't have to depend on `Game`, they could be arranged in a linked list. Each `Frame` could hold pointers to its next and previous `Frames`. To get the score from a `Frame`, the `Frame` would look backward to get the score of the previous `Frame` and forward for any spare or strike balls it needs.

RSK: Okay, I'm feeling kind of dumb because I can't visualize this. Show me some code.

RCM: Right. So we need a test case first.

RSK: For `Game` or another test for `Frame`?

RCM: I think we need one for `Game`, since it's `Game` that will build the `Frames` and hook them up to each other.

RSK: Do you want to stop what we're doing on `Frame` and do a mental `longjump` to `Game`, or do you just want to have a `MockGame` object that does just what we need to get `Frame` working?

RCM: No, let's stop working on `Frame` and start working on `Game`. The test cases in `Game` should prove that we need the linked list of `Frames`.

RSK: I'm not sure how they'll show the need for the list. I need code.

RCM: (types)

```
//TestGame.java-----------------------------------------
import junit.framework.*;

public class TestGame extends TestCase
{
  public TestGame(String name)
  {
    super(name);
  }

  public void testOneThrow()
  {
    Game g = new Game();
    g.add(5);
    assertEquals(5, g.score());
  }

}
```

RCM: Does that look reasonable?

RSK: Sure, but I'm still looking for proof for this list of `Frames`.

RCM: Me too. Let's keep following these test cases and see where they lead.

```
//Game.java---------------------------------
public class Game
{
  public int score()
  {
    return 0;
  }

  public void add(int pins)
  {
  }
}
```

RCM: OK, this compiles and fails the test. Now let's make it pass.

```
//Game.java---------------------------------
public class Game
{
  public int score()
  {
    return itsScore;
  }
```

```
  public void add(int pins)
  {
    itsScore += pins;
  }
  private int itsScore = 0;
}
```

RCM: This passes. Good.

RSK: I can't disagree with it, but I'm still looking for this great proof of the need for a linked list of `Frame` objects. That's what led us to `Game` in the first place.

RCM: Yeah, that's what I'm looking for too. I fully expect that once we start injecting spare and strike test cases, we'll have to build `Frames` and tie them together in a linked list. But I don't want to build that until the code forces us to.

RSK: Good point. Let's keep going in small steps on `Game`. What about another test that tests two throws but with no spare?

RCM: OK, that should pass right now. Let's try it.

```
//TestGame.java-----------------------------------------

  public void testTwoThrowsNoMark()
  {
    Game g = new Game();
    g.add(5);
    g.add(4);
    assertEquals(9, g.score());
  }
```

RCM: Yep, that one passes. Now let's try four balls with no marks.

RSK: Well that will pass, too. I didn't expect this. We can keep adding throws, and we don't ever even need a `Frame`. But we haven't done a spare or a strike yet. Maybe that's when we'll have to make one.

RCM: That's what I'm counting on. However, consider this test case:

```
//TestGame.java-----------------------------------------
public void testFourThrowsNoMark()
  {
    Game g = new Game();
    g.add(5);
    g.add(4);
    g.add(7);
    g.add(2);
    assertEquals(18, g.score());
    assertEquals(9,  g.scoreForFrame(1));
    assertEquals(18, g.scoreForFrame(2));
  }
```

RCM: Does this look reasonable?

RSK: It sure does. I forgot that we have to be able to show the score in each frame. Ah, our sketch of the score card was serving as a coaster for my Diet Coke. Yeah, that's why I forgot.

RCM: (sigh) OK, first let's make this test case fail by adding the `scoreForFrame` method to `Game`.

```
//Game.java----------------------------------

  public int scoreForFrame(int frame)
  {
    return 0;
  }
```

RCM: Great, this compiles and fails. Now, how do we make it pass?

RSK: We can start making `Frame` objects, but is that the simplest thing that will get the test to pass?

RCM: No, actually, we could just create an array of integers in the `Game`. Each call to `add` would append a new integer onto the array. Each call to `scoreForFrame` will just work forward through the array and calculate the score.

```
//Game.java----------------------------------
public class Game
{
  public int score()
  {
    return itsScore;
  }

  public void add(int pins)
  {
    itsThrows[itsCurrentThrow++]=pins;
    itsScore += pins;
  }

  public int scoreForFrame(int frame)
  {
    int score = 0;
    for ( int ball = 0;
      frame > 0 && (ball < itsCurrentThrow);
      ball+=2, frame--)
    {
      score += itsThrows[ball] + itsThrows[ball+1];
    }
    return score;
  }
  private int itsScore = 0;
  private int[] itsThrows = new int[21];
  private int itsCurrentThrow = 0;
}
```

RCM: (very satisfied with himself) There, that works.

RSK: Why the magic number 21?

RCM: That's the maximum possible number of throws in a game.

RSK: Yuck. Let me guess, in your youth you were a Unix hacker and prided yourself on writing an entire application in one statement that nobody else could decipher.

`scoreForFrame()` needs to be refactored to be more communicative. But before we consider refactoring, let me ask another question. Is `Game` the best place for this method? In my mind, `Game`

is violating the Single Responsibility Principle.[1] It is accepting throws *and* it knows how to score for each frame. What would you think about a `Scorer` object?

RCM: (makes a rude oscillating gesture with his hand) I don't know where the functions live now; right now I'm interested in getting the scoring stuff to work. Once we've got that all in place, *then* we can debate the values of the SRP.

However, I see your point about the Unix hacker stuff; let's try to simplify that loop.

```
public int scoreForFrame(int theFrame)
{
  int ball = 0;
  int score=0;
  for (int currentFrame = 0;
       currentFrame < theFrame;
       currentFrame++)
  {
   score += itsThrows[ball++] + itsThrows[ball++];
  }

  return score;
}
```

RCM: That's a little better, but there are side effects in the `score+=` expression. They don't matter here because it doesn't matter which order the two addend expressions are evaluated in. (Or does it? Is it possible that the two increments could be done before either array operation?)

RSK: I suppose we could do an experiment to verify that there aren't any side effects, but that function isn't going to work with spares and strikes. Should we keep trying to make it more readable, or should we push further on its functionality?

RCM: The experiment might only have meaning on certain compilers. Other compilers might use different evaluation orders. I don't know if this is an issue or not, but let's get rid of the potential order dependency and then push on with more test cases.

```
public int scoreForFrame(int theFrame)
{
   int ball = 0;
   int score=0;
   for (int currentFrame = 0;
        currentFrame < theFrame;
        currentFrame++)
   {
     int firstThrow = itsThrows[ball++];
     int secondThrow = itsThrows[ball++];
     score += firstThrow + secondThrow;
   }

   return score;
}
```

RCM: OK, next test case. Let's try a spare.

1. See "SRP: The Single-Responsibility Principle" on page 95.

```
public void testSimpleSpare()
{
  Game g = new Game();
}
```

RCM: I'm tired of writing this. Let's refactor the test and put the creation of the game in a `setUp` function.

```
//TestGame.java-----------------------------------------
import junit.framework.*;

public class TestGame extends TestCase
{
  public TestGame(String name)
  {
    super(name);
  }

  private Game g;

  public void setUp()
  {
    g = new Game();
  }

 public void testOneThrow()
 {
  g.add(5);
  assertEquals(5, g.score());
 }

  public void testTwoThrowsNoMark()
  {
    g.add(5);
    g.add(4);
    assertEquals(9, g.score());
  }

  public void testFourThrowsNoMark()
  {
    g.add(5);
    g.add(4);
    g.add(7);
    g.add(2);
    assertEquals(18, g.score());
    assertEquals(9,  g.scoreForFrame(1));
    assertEquals(18, g.scoreForFrame(2));
  }

  public void testSimpleSpare()
  {
  }
}
```

RCM: That's better, now let's write the spare test case.

```
public void testSimpleSpare()
{
  g.add(3);
  g.add(7);
  g.add(3);
  assertEquals(13, g.scoreForFrame(1));
}
```

RCM: OK, that test case fails. Now we need to make it pass.

RSK: I'll drive.

```
public int scoreForFrame(int theFrame)
{
  int ball = 0;
  int score=0;
  for (int currentFrame = 0;
       currentFrame < theFrame;
       currentFrame++)
  {
    int firstThrow = itsThrows[ball++];
    int secondThrow = itsThrows[ball++];

    int frameScore = firstThrow + secondThrow;
    // spare needs next frames first throw
    if ( frameScore == 10 )
      score += frameScore + itsThrows[ball++];
    else
      score += frameScore;
  }

  return score;
}
```

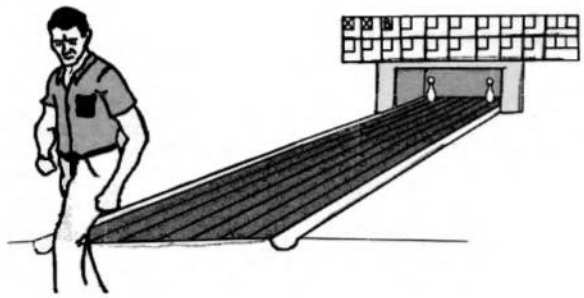
RSK: Yee-HA! That works!

RCM: (grabbing the keyboard) OK, but I think the increment of ball in the `frameScore==10` case shouldn't be there. Here's a test case that proves my point.

```
public void testSimpleFrameAfterSpare()
{
  g.add(3);
  g.add(7);
  g.add(3);
  g.add(2);
  assertEquals(13, g.scoreForFrame(1));
  assertEquals(18, g.score());
}
```

RCM: Ha! See, that fails. Now if we just take out that pesky extra increment ...

```
if ( frameScore == 10 )
  score += frameScore + itsThrows[ball];
```



RCM: Uh . . . It still fails . . . Could it be that the `score` method is wrong? I'll test that by changing the test case to use `scoreForFrame(2)`.

```
public void testSimpleFrameAfterSpare()
{
  g.add(3);
  g.add(7);
  g.add(3);
  g.add(2);
  assertEquals(13, g.scoreForFrame(1));
  assertEquals(18, g.scoreForFrame(2));
}
```

RCM: Hmmmm . . . That passes. The `score` method must be messed up. Let's look at it.

```
public int score()
{
  return itsScore;
}

public void add(int pins)
{
  itsThrows[itsCurrentThrow++]=pins;
  itsScore += pins;
}
```

RCM: Yeah, that's wrong. The `score` method is just returning the sum of the pins, not the proper score. What we need `score` to do is call `scoreForFrame()` with the current frame.

RSK: We don't know what the current frame is. Let's add that message to each of our current tests, one at a time, of course.

RCM: Right.

```
//TestGame.java-----------------------------------------
  public void testOneThrow()
  {
    g.add(5);
    assertEquals(5, g.score());
    assertEquals(1, g.getCurrentFrame());
  }
```

```
//Game.java---------------------------------
  public int getCurrentFrame()
  {
    return 1;
  }
```

RCM: OK, that works. But it's stupid. Let's do the next test case.

```
public void testTwoThrowsNoMark()
{
  g.add(5);
  g.add(4);
  assertEquals(9, g.score());
  assertEquals(1, g.getCurrentFrame());
}
```

RCM: That one's uninteresting, let's try the next.

```
public void testFourThrowsNoMark()
{
  g.add(5);
  g.add(4);
  g.add(7);
  g.add(2);
  assertEquals(18, g.score());
  assertEquals(9,  g.scoreForFrame(1));
  assertEquals(18, g.scoreForFrame(2));
  assertEquals(2, g.getCurrentFrame());
}
```

RCM: This one fails. Now let's make it pass.

RSK: I think the algorithm is trivial. Just divide the number of throws by two, since there are two throws per frame. Unless we have a strike ... but we don't have strikes yet, so let's ignore them here too.

RCM: (flails around adding and subtracting 1 until it works)[2]

```
public int getCurrentFrame()
{
  return 1 + (itsCurrentThrow-1)/2;
}
```

RCM: That isn't very satisfying.

RSK: What if we don't calculate it each time? What if we adjust a `currentFrame` member variable after each throw?

RCM: OK, let's try that.

```
//Game.java---------------------------------
  public int getCurrentFrame()
  {
    return itsCurrentFrame;
  }
```

2. Dave Thomas and Andy Hunt call this "programming by coincidence."

```
public void add(int pins)
{
  itsThrows[itsCurrentThrow++]=pins;
  itsScore += pins;
  if (firstThrow == true)
  {
    firstThrow = false;
    itsCurrentFrame++;
  }
  else
  {
    firstThrow=true;;
  }
}

private int itsCurrentFrame = 0;
private boolean firstThrow = true;
```

RCM: OK, this works. But it also implies that the current frame is the frame of the last ball thrown, not the frame that the next ball will be thrown into. As long as we remember that, we'll be fine.

RSK: I don't have that good of a memory, so let's make it more readable. But before we go screwing around with it some more, let's pull that code out of `add()` and put it in a private member function called `adjustCurrentFrame()` or something.

RCM: OK, that sounds good.

```
public void add(int pins)
{
  itsThrows[itsCurrentThrow++]=pins;
  itsScore += pins;
  adjustCurrentFrame();
}

private void adjustCurrentFrame()
{
  if (firstThrow == true)
  {
    firstThrow = false;
    itsCurrentFrame++;
  }
  else
  {
    firstThrow=true;;
  }
}
```

RCM: Now let's change the variable and function names to be more clear. What should we call `itsCurrentFrame`?

RSK: I kind of like that name. I don't think we're incrementing it in the right place, though. The current frame, to me, is the frame number that I'm throwing in. So it should get incremented right after the last throw in a frame.

RCM: I agree. Let's change the test cases to reflect that, then we'll fix `adjustCurrentFrame`.

```
//TestGame.java-----------------------------------------
  public void testTwoThrowsNoMark()
  {
    g.add(5);
    g.add(4);
    assertEquals(9, g.score());
    assertEquals(2, g.getCurrentFrame());
  }

  public void testFourThrowsNoMark()
  {
    g.add(5);
    g.add(4);
    g.add(7);
    g.add(2);
    assertEquals(18, g.score());
    assertEquals(9,  g.scoreForFrame(1));
    assertEquals(18, g.scoreForFrame(2));
    assertEquals(3, g.getCurrentFrame());
  }

//Game.java---------------------------------------------
  private void adjustCurrentFrame()
  {
    if (firstThrow == true)
    {
      firstThrow = false;
    }
    else
    {
      firstThrow=true;
      itsCurrentFrame++;
    }
  }

  private int itsCurrentFrame = 1;
}
```

RCM: OK, that's working. Now let's test `getCurrentFrame` in the two spare cases.

```
public void testSimpleSpare()
{
  g.add(3);
  g.add(7);
  g.add(3);
  assertEquals(13, g.scoreForFrame(1));
  assertEquals(2, g.getCurrentFrame());
}

public void testSimpleFrameAfterSpare()
{
  g.add(3);
  g.add(7);
  g.add(3);
  g.add(2);
```

```
    assertEquals(13, g.scoreForFrame(1));
    assertEquals(18, g.scoreForFrame(2));
    assertEquals(3, g.getCurrentFrame());
}
```

RCM: This works. Now, back to the original problem. We need `score` to work. We can now write `score` to call `scoreForFrame(getCurrentFrame()-1)`.

```
public void testSimpleFrameAfterSpare()
{
  g.add(3);
  g.add(7);
  g.add(3);
  g.add(2);
  assertEquals(13, g.scoreForFrame(1));
  assertEquals(18, g.scoreForFrame(2));
  assertEquals(18, g.score());
  assertEquals(3, g.getCurrentFrame());
}

//Game.java---------------------------------
  public int score()
  {
    return scoreForFrame(getCurrentFrame()-1);
  }
```

RCM: This fails the `TestOneThrow` test case. Let's look at it.

```
public void testOneThrow()
{
  g.add(5);
  assertEquals(5, g.score());
  assertEquals(1, g.getCurrentFrame());
}
```

RCM: With only one throw, the first frame is incomplete. The score method is calling `scoreForFrame(0)`. This is yucky.

RSK: Maybe, maybe not. Who are we writing this program for, and who is going to be calling `score()`? Is it reasonable to assume that it won't get called on an incomplete frame?

RCM: Yeah, but it bothers me. To get around this, we have take the `score` out of the `testOneThrow` test case. Is that what we want to do?

RSK: We could. We could even eliminate the entire `testOneThrow` test case. It was used to ramp us up to the test cases of interest. Does it really serve a useful purpose now? We still have coverage in all of the other test cases.

RCM: Yeah, I see your point. OK, out it goes. (edits code, runs test, gets green bar) Ahhh, that's better.

Now, we'd better work on the strike test case. After all, we want to see all those `Frame` objects built into a linked list, don't we? (snicker)

```
public void testSimpleStrike()
{
  g.add(10);
```



```
  g.add(3);
  g.add(6);
  assertEquals(19, g.scoreForFrame(1));
  assertEquals(28, g.score());
  assertEquals(3, g.getCurrentFrame());
}
```

RCM: OK, this compiles and fails as predicted. Now we need to make it pass.

```
//Game.java---------------------------------
public class Game
{
  public void add(int pins)
  {
    itsThrows[itsCurrentThrow++]=pins;
    itsScore += pins;
    adjustCurrentFrame(pins);
  }

  private void adjustCurrentFrame(int pins)
  {
    if (firstThrow == true)
    {
      if( pins == 10 ) // strike
        itsCurrentFrame++;
      else
        firstThrow = false;
    }
    else
    {
      firstThrow=true;
      itsCurrentFrame++;
    }
  }

  public int scoreForFrame(int theFrame)
  {
    int ball = 0;
    int score=0;
    for (int currentFrame = 0;
         currentFrame < theFrame;
         currentFrame++)
```

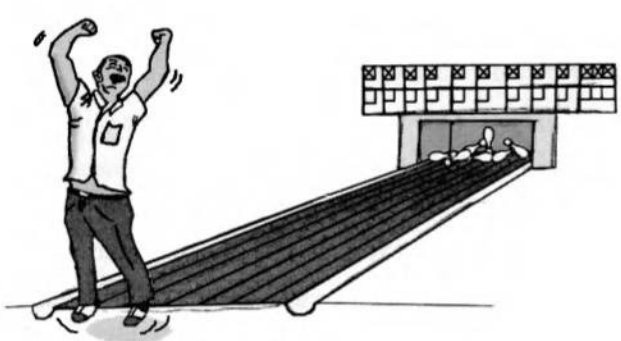
```
    {
      int firstThrow = itsThrows[ball++];
      if (firstThrow == 10)
      {
         score += 10 + itsThrows[ball] + itsThrows[ball+1];
      }
      else
      {
         int secondThrow = itsThrows[ball++];

         int frameScore = firstThrow + secondThrow;
         // spare needs next frames first throw
         if ( frameScore == 10 )
           score += frameScore + itsThrows[ball];
         else
           score += frameScore;
      }

    }

    return score;
  }
  private int itsScore = 0;
  private int[] itsThrows = new int[21];
  private int itsCurrentThrow = 0;
  private int itsCurrentFrame = 1;
  private boolean firstThrow = true;
}
```

RCM: OK, that wasn't too hard. Let's see if it can score a perfect game.



```
public void testPerfectGame()
{
  for (int i=0; i<12; i++)
  {
    g.add(10);
  }
  assertEquals(300, g.score());
  assertEquals(10, g.getCurrentFrame());
}
```

RCM: Urg, it's saying the score is 330. Why would that be?

RSK: Because the current frame is getting incremented all the way to 12.

RCM: Oh! We need to limit it to 10.

```
private void adjustCurrentFrame(int pins)
{
  if (firstThrow == true)
  {
    if( pins == 10 ) // strike
      itsCurrentFrame++;
    else
      firstThrow = false;
  }
  else
  {
    firstThrow=true;
    itsCurrentFrame++;
  }
  itsCurrentFrame = Math.min(10, itsCurrentFrame);
}
```

RCM: Damn, now it's saying that the score is 270. What's going on?

RSK: Bob, the `score` function is subtracting one from `getCurrentFrame`, so it's giving you the score for frame 9, not 10.

RCM: What? You mean I should limit the current frame to 11 not 10? I'll try it.

```
itsCurrentFrame = Math.min(11, itsCurrentFrame);
```

RCM: OK, so now it gets the score correct but fails because the current frame is 11 and not 10. Ick! This current frame thing is a pain in the butt. We want the current frame to be the frame the player is throwing into, but what does that mean at the end of the game?

RSK: Maybe we should go back to the idea that the current frame is the frame of the last ball thrown.

RCM: Or maybe we need to come up with the concept of the last *completed* frame? After all, the score of the game at any point in time is the score in the last completed frame.

RSK: A completed frame is a frame that you can write the score into, right?

RCM: Yes, a frame with a spare in it completes after the next ball. A frame with a strike in it completes after the next two balls. A frame with no mark completes after the second ball in the frame.

Wait a minute . . . We are trying to get the `score()` method to work, right? All we need to do is force `score()` to call `scoreForFrame(10)` if the game is complete.

RSK: How do we know if the game is complete?

RCM: If `adjustCurrentFrame` ever tries to increment `itsCurrentFrame` past the tenth frame, then the game is complete.

RSK: Wait. All you are saying is that if `getCurrentFrame` returns 11, the game is complete. That's the way the code works now!

RCM: Hmm. You mean we should change the test case to match the code?

```
public void testPerfectGame()
{
  for (int i=0; i<12; i++)
  {
    g.add(10);
  }
  assertEquals(300, g.score());
  assertEquals(11, g.getCurrentFrame());
}
```

RCM: Well, that works. I suppose it's no worse than `getMonth` returning 0 for January. But I still feel uneasy about it.

RSK: Maybe something will occur to us later. Right now, I think I see a bug. May I?" (grabs keyboard)

```
public void testEndOfArray()
{
  for (int i=0; i<9; i++)
  {
    g.add(0);
    g.add(0);
  }
  g.add(2);
  g.add(8); // 10th frame spare
  g.add(10); // Strike in last position of array.
  assertEquals(20, g.score());
}
```

RSK: Hmm. That doesn't fail. I thought since the 21st position of the array was a strike, the scorer would try to add the 22nd and 23rd positions to the score. But I guess not.

RCM: Hmm, you are still thinking about that scorer object aren't you. Anyway, I see what you were getting at, but since `score` never calls `scoreForFrame` with a number larger than 10, the last strike is not actually counted as a strike. It's just counted as a 10 to complete the last spare. We never walk beyond the end of the array.

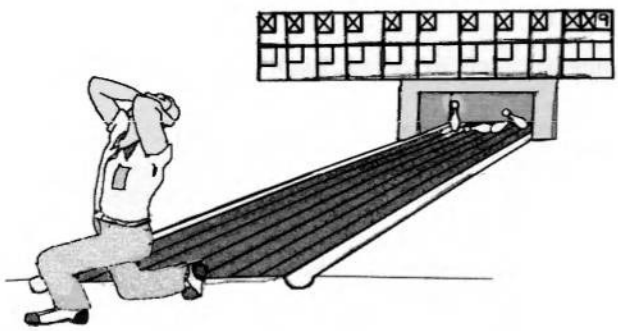
RSK: OK, let's pump our original score card into the program.

```
public void testSampleGame()
{
  g.add(1);
  g.add(4);
  g.add(4);
  g.add(5);
  g.add(6);
  g.add(4);
  g.add(5);
  g.add(5);
  g.add(10);
  g.add(0);
  g.add(1);
  g.add(7);
  g.add(3);
  g.add(6);
  g.add(4);
  g.add(10);
```

```
    g.add(2);
    g.add(8);
    g.add(6);
    assertEquals(133, g.score());
}
```

RSK: Well, that works. Are there any other test cases that you can think of?

RCM: Yeah, let's test a few more boundary conditions— how about the poor schmuck who throws 11 strikes and then a final 9?



```
public void testHeartBreak()
{
  for (int i=0; i<11; i++)
   g.add(10);
  g.add(9);
  assertEquals(299, g.score());
}
```

RCM: That works. OK, how about a tenth frame spare?

```
public void testTenthFrameSpare()
{
  for (int i=0; i<9; i++)
    g.add(10);
  g.add(9);
  g.add(1);
  g.add(1);
  assertEquals(270, g.score());
}
```

RCM: (staring happily at the green bar) That works too. I can't think of any more, can you.

RSK: No, I think we've covered them all. Besides, I really want to refactor this mess. I still see the `scorer` object in there somewhere.

RCM: OK, well, the `scoreForFrame` function is pretty messy. Let's consider it.

```
public int scoreForFrame(int theFrame)
{
  int ball = 0;
  int score=0;
```

```
  for (int currentFrame = 0;
       currentFrame < theFrame;
     currentFrame++)
  {
    int firstThrow = itsThrows[ball++];
    if (firstThrow == 10)
    {
      score += 10 + itsThrows[ball] + itsThrows[ball+1];
    }
    else
    {
      int secondThrow = itsThrows[ball++];

      int frameScore = firstThrow + secondThrow;
      // spare needs next frames first throw
      if ( frameScore == 10 )
        score += frameScore + itsThrows[ball];
      else
        score += frameScore;
    }

  }

  return score;
}
```

RCM: I'd really like to extract the body of that `else` clause into a separate function named `handleSecondThrow`, but I can't because it uses `ball`, `firstThrow`, and `secondThrow` local variables.

RSK: We could turn those locals into member variables.

RCM: Yeah, that kind of reenforces your notion that we'll be able to pull the scoring out into its own `scorer` object. OK, let's give that a try.

RSK: (grabs keyboard)

```
private void adjustCurrentFrame(int pins)
{
  if (firstThrowInFrame == true)
  {
    if( pins == 10 ) // strike
      itsCurrentFrame++;
    else
      firstThrowInFrame = false;
  }
  else
  {
    firstThrowInFrame=true;
    itsCurrentFrame++;
  }
  itsCurrentFrame = Math.min(11, itsCurrentFrame);
}
```